

YOLO项目运行解析-教学文档、说明

本文及其所有项目由本人原创优化训练。

本项目仅限个人学习与非商业用途，严禁任何形式的二次售卖、转发、转授权或用于任何商业行为。

YOLOv8开源遵守AGPL-3.0 许可证。本项目收取的费用仅包含项目的UI系统编写+系统调试、训练、bug解决、教学+微调改进的辛苦费用，仅限个人学习。

本项目中的UI系统已申请版权保护。并已嵌入特殊追踪标识。

一旦用户下载或使用本项目，视为已知悉并同意以上声明内容。

启动篇

项目启动十分简单。只需运行

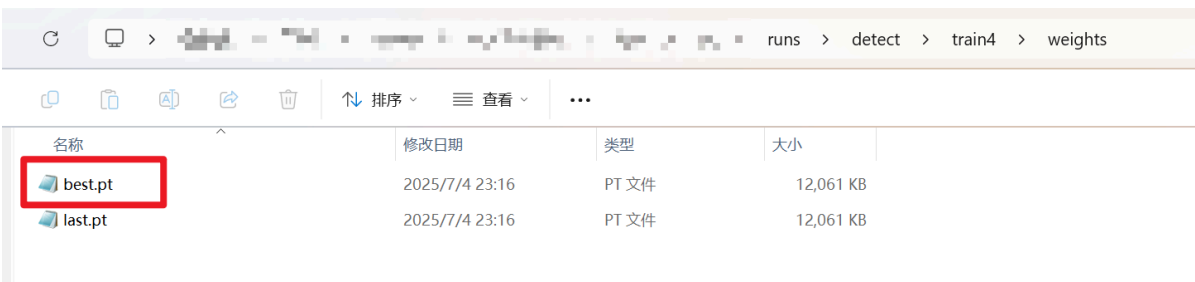
youi文件夹下的main.py即可，前提条件需要安装项目所需的依赖包。



1.模型路径:

选择训练好的best.pt文件

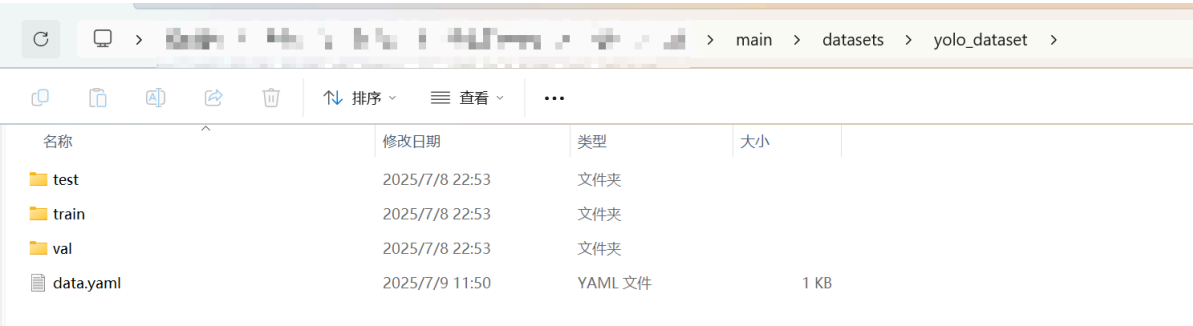
通常在根目录下的runs文件夹内



2.输入路径

按照要求输入图片、图片文件夹或者视频即可

数据集一般在main文件夹内的datasets文件夹中。



3.输出目录

选择一个方便的保存结果的目录即可。

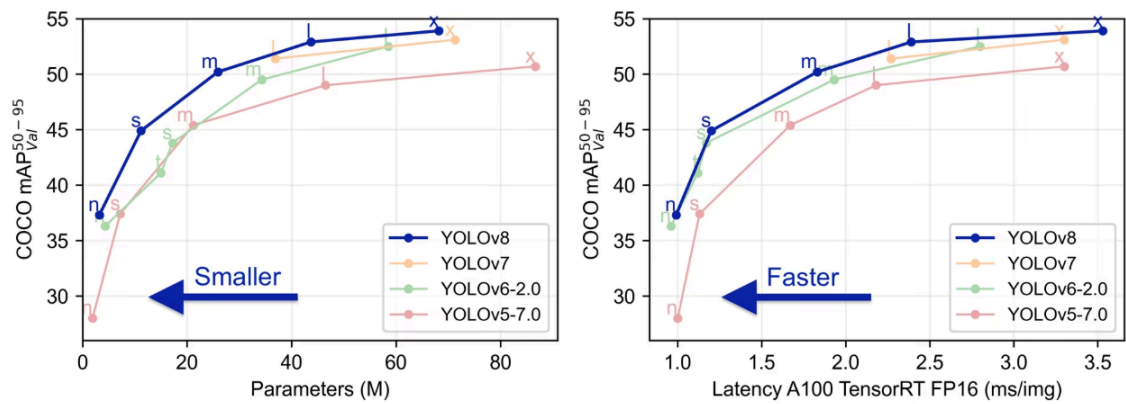
原理篇

YOLOv8的基本原理

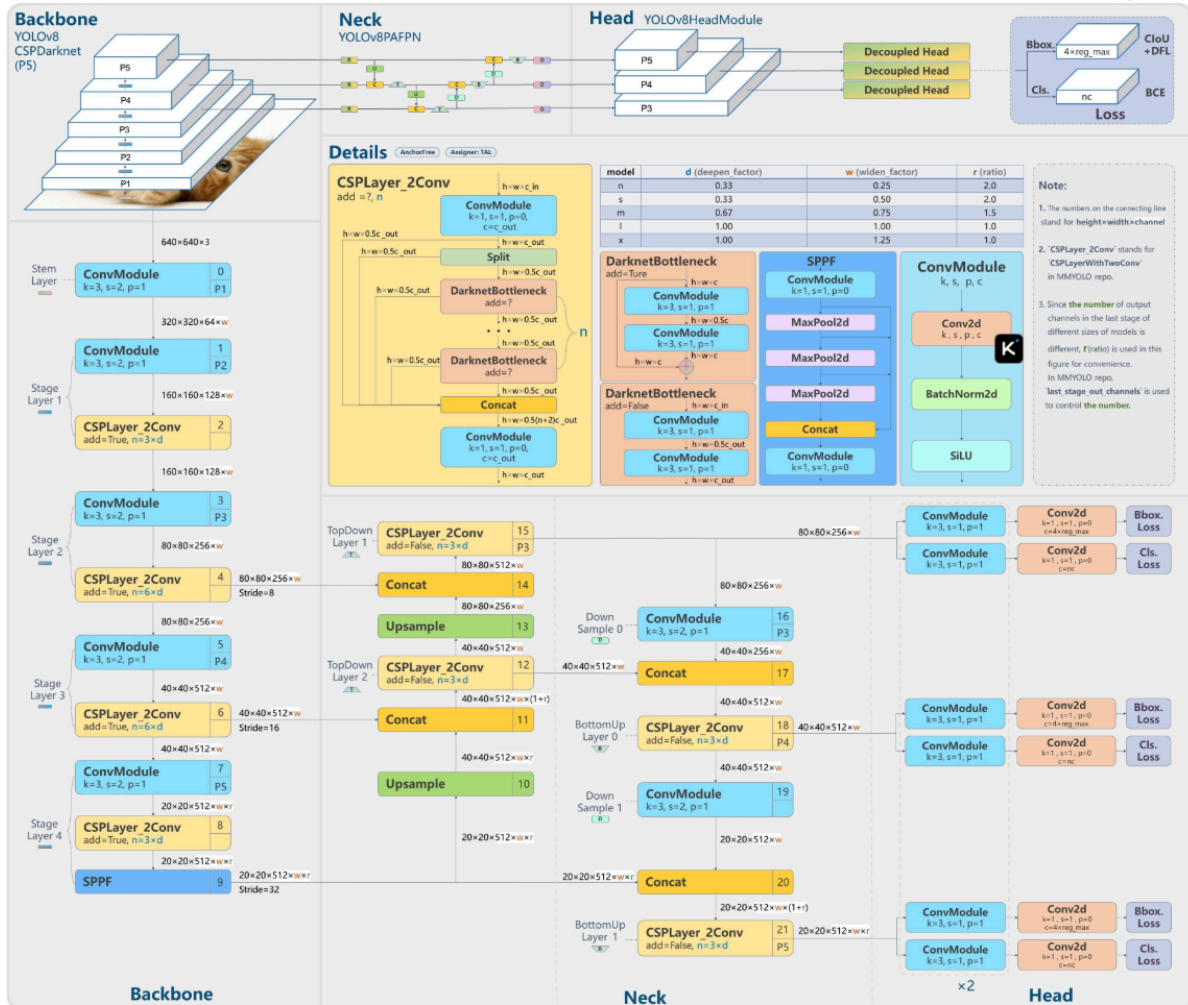
YOLOv8 是新一代实时目标检测模型，具备如下优势：

- **速度快**：推理速度提升明显；
- **准确率高**：支持 Anchor-Free 架构；
- **支持分类/检测/分割/姿态多任务**；
- 本项目使用 YOLOv8 的 Detection 分支，训练时每类表情均标注为独立目标。

YOLOv8 于 2023 年 1 月 10 日发布，在准确性和速度方面具有尖端性能。在以往YOLO 版本的基础上，YOLOv8 引入了新的功能和优化，使其成为广泛应用中各种物体检测任务的理想选择。



YOLOv8原理图如下：



运行篇

python推荐版本: 3.10

安装pytorch

[PyTorch](#)

执行下文命令

NOTE: Latest PyTorch requires Python 3.9 or later.

PyTorch Build	Stable (2.7.0)			Preview (Nightly)	
Your OS	Linux		Mac		Windows
Package	Pip		LibTorch		Source
Language	Python			C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.6	CUDA 12.8	ROCm 6.3	CPU

Run this Command:

```
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
```

安装库

执行命令

```
pip install requirements.txt
```

制作数据集-举个例子

采用 YOLO 格式的数据集结构如下：

```
dataset/  
├─ images/  
│   └─ train/  
│       └─ val/  
├─ labels/  
│   └─ train/  
│       └─ val/
```

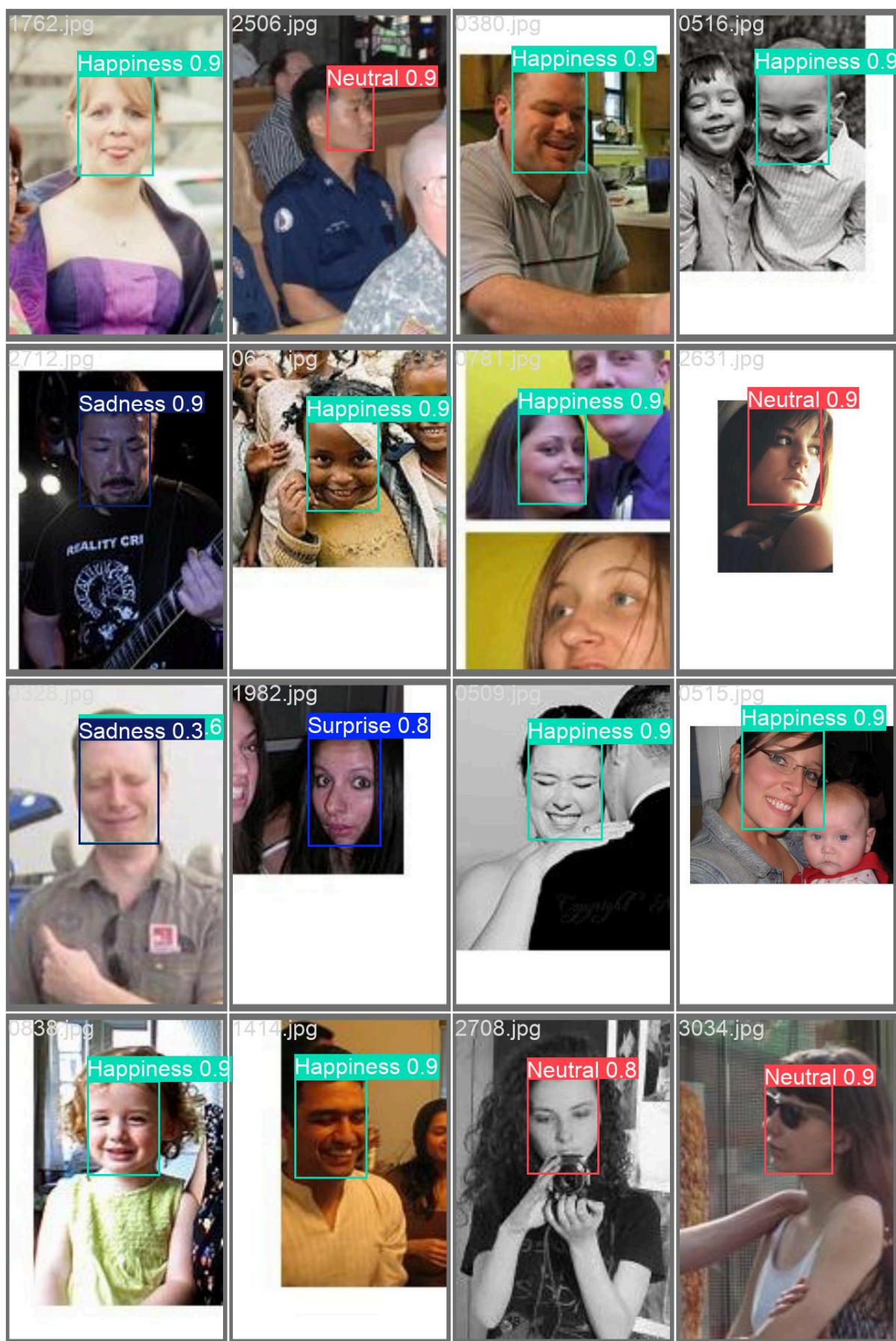
每张图像有对应的 `.txt` 文件，内容格式为：

```
4 0.5096721233576642 0.352838390077821 0.3947600423357664 0.31825755058365757
```

表情分类包括（可自定义）：

```
nc: 7  
names: ["Surprise", "Fear", "Disgust", "Happiness", "Sadness", "Anger",  
"Neutral"]
```

惊讶、恐惧、厌恶、快乐、悲伤、愤怒、平静



训练项目:训练命令要自己调，数据集路径，批次，模型等等

使用 CLI 工具:

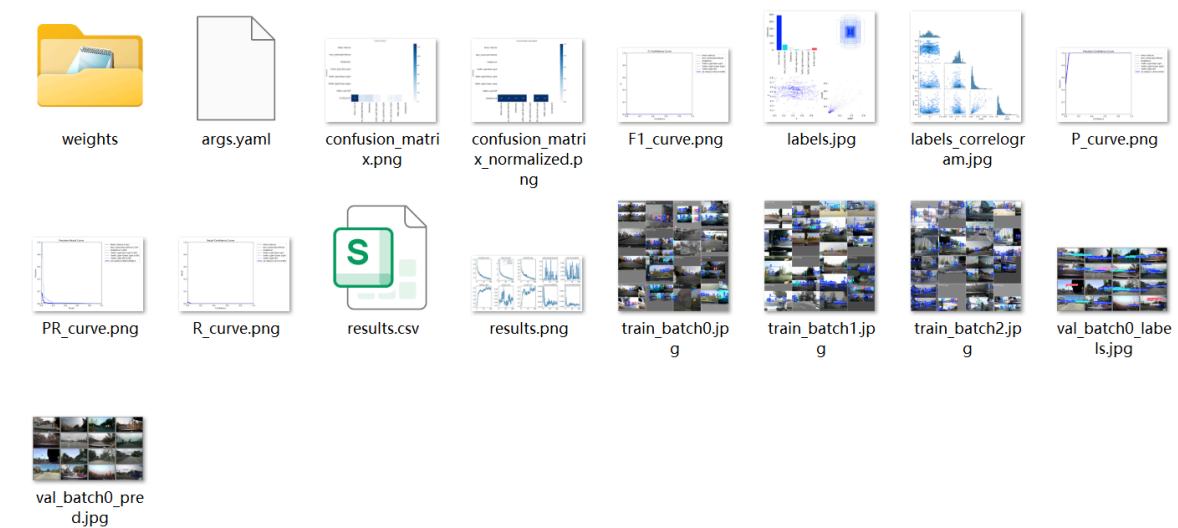
注意datasets/expression/loopy.yaml【这个地址要替换为数据集的yaml地址】。

```
yolo detect train data=datasets/expression/loopy.yaml model=yolov8n.yaml
pretrained=yolov8n.pt epochs=100 batch=16 lr0=0.001
```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
98/100	2.126	0.5488	0.2594	0.9009	15	640: 100%	767/767	[01:48<00:00, 7.09it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	96/96	[00:20<00:00, 4.80it/s]
	all	3068	3068	0.852	0.776	0.867	0.721		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
99/100	2.126	0.5457	0.2528	0.9006	15	640: 100%	767/767	[01:48<00:00, 7.10it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	96/96	[00:19<00:00, 4.92it/s]
	all	3068	3068	0.852	0.775	0.867	0.721		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
100/100	2.126	0.5423	0.2509	0.8966	15	640: 100%	767/767	[01:48<00:00, 7.10it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	96/96	[00:19<00:00, 4.85it/s]
	all	3068	3068	0.851	0.78	0.867	0.72		
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%			
all	30	179	0.00187	0.0228	0.00195	0.000425	1/1	[00:00<00:00, 5.89it/s]	
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
100/500	2.696	1.287	1.168	1.211	185	640: 100%	6/6	[00:00<00:00, 7.11it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	1/1	[00:00<00:00, 5.88it/s]
	all	30	179	0.171	0.00394	0.00182	0.000304		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
101/500	2.696	1.272	1.148	1.195	144	640: 100%	6/6	[00:00<00:00, 7.04it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	1/1	[00:00<00:00, 5.58it/s]
	all	30	179	0.00258	0.00787	0.00142	0.00026		
Non-motorized Vehicle	12	18	0.00667	0.167	0.00419	0.00217			
Pedestrian	8	23	0	0	0	0			
Traffic Light-Red Light	2	4	0	0	0	0			
Traffic Light-Green Light	1	1	0	0	0	0			
Traffic Light-Off	3	6	0	0	0	0			

Speed: 0.1ms preprocess, 1.0ms inference, 0.0ms loss, 1.5ms postprocess per image

训练成果后，结果如下



如果是分类任务：

```
yolo classify train data=datasets/expression/loopy.yaml model=yolov8n-cls.yaml
pretrained=yolov8n-cls.pt epochs=100 batch=16 lr0=0.001
```

五大训练命令备注：

✅ **yolo detect**：目标检测任务

```
yolo detect train data=dataset.yaml model=yolov8n.yaml pretrained=yolov8n.pt
epochs=100 imgsz=640
```

✅ **yolo segment**：实例分割任务

```
yolo segment train data=dataset.yaml model=yolov8n-seg.yaml pretrained=yolov8n-seg.pt epochs=100 imgsz=640
```

✅ yolo classify: 图像分类任务

```
yolo classify train data=dataset.yaml model=yolov8n-cls.yaml pretrained=yolov8n-cls.pt epochs=100 imgsz=224
```

✅ yolo pose: 人体姿态估计任务

```
yolo pose train data=dataset.yaml model=yolov8n-pose.yaml pretrained=yolov8n-pose.pt epochs=100 imgsz=640
```

✅ yolo track: 多目标跟踪任务（基于检测结果）

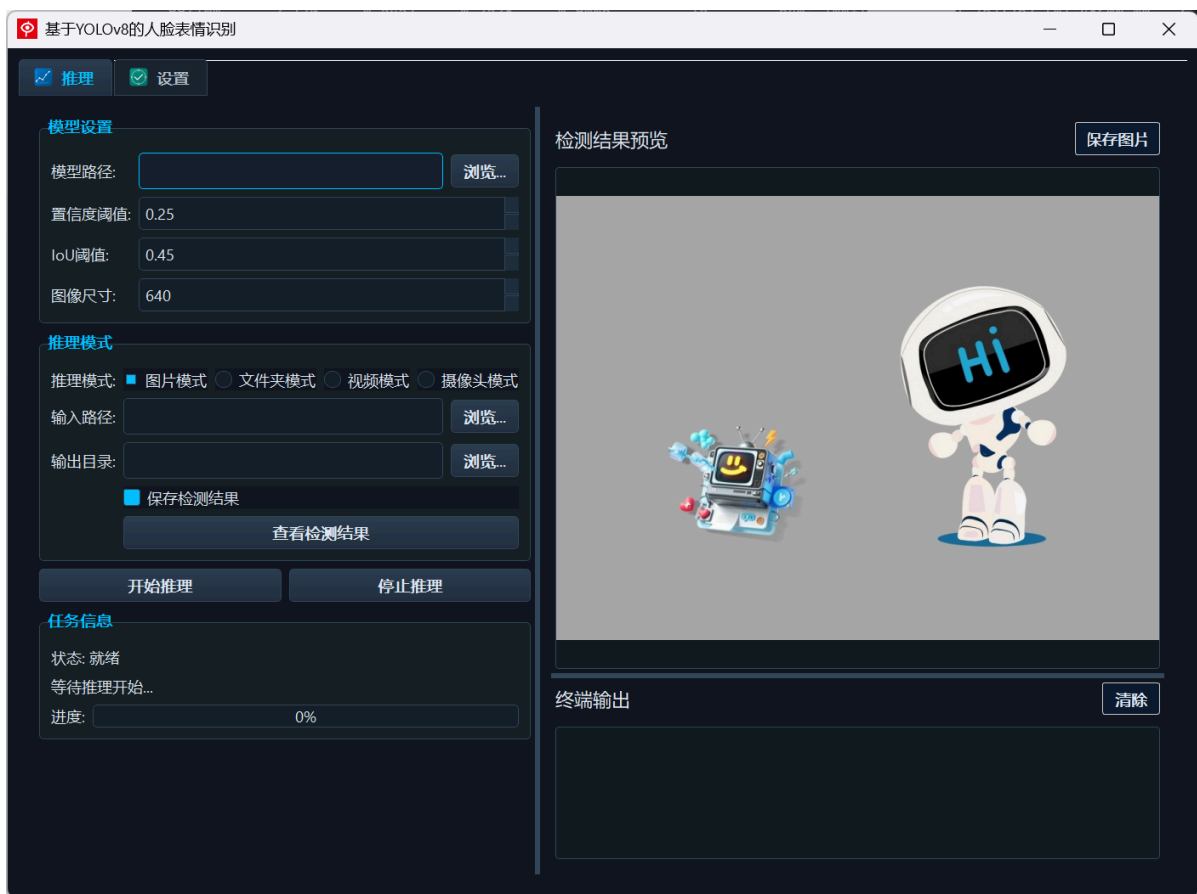
```
yolo track predict model=yolov8n.pt source=video.mp4 tracker=bytetrack.yaml
```

启动YOLOUI页面

执行命令

```
python ./youi/main.py
```

启动项目如下。



项目结构

```
-.idea
|   └─inspectionProfiles
├─main
|   └─.idea
|       └─inspectionProfiles
|   └─build
|       └─bdist.win-amd64
|       └─lib
|           └─ultralytics
|               └─cfg
|                   └─models
|                       └─rt-detr
|                       └─v3
|                       └─v5
|                       └─v6
|                       └─v8
|                   └─trackers
|               └─data
|               └─engine
|               └─hub
|               └─models
|                   └─fastsam
|                   └─nas
|                   └─rtdetr
|                   └─sam
|                   └─modules
|                   └─utils
|                   └─yolo
```

```

├── classify
├── detect
├── pose
├── segment
├── nn
├──   ├── modules
├──   ├── trackers
├──   ├── utils
├──   ├──   ├── utils
├──   │   ├── callbacks
├──   │   └── yolo
├──   │       ├── cfg
├──   │       ├── data
├──   │       ├── engine
├──   │       ├── utils
├──   │       └── v8
├── datasets
├──   ├── jiaotong
├──   │   ├── train
├──   │   │   ├── images
├──   │   │   └── labels
├──   │   └── val
├──   │       ├── images
├──   │       └── labels
├── dist
├── docker
├── docs
├──   ├── assets
├──   ├── guides
├──   ├── help
├──   ├── hub
├──   │   ├── app
├──   ├── integrations
├──   ├── models
├──   ├── modes
├──   ├── overrides
├──   │   ├── partials
├──   ├── reference
├──   │   ├── cfg
├──   │   ├── data
├──   │   ├── engine
├──   │   ├── hub
├──   │   ├── models
├──   │   │   ├── fastsam
├──   │   │   ├── nas
├──   │   │   ├── rtdetr
├──   │   │   ├── sam
├──   │   │   │   ├── modules
├──   │   │   ├── utils
├──   │   └── yolo
├──   │       ├── classify
├──   │       ├── detect
├──   │       ├── pose
├──   │       └── segment
├──   ├── nn
├──   │   ├── modules

```

```

| | | | trackers
| | | | | utils
| | | | | utils
| | | | | callbacks
| | | | stylesheets
| | | | tasks
| | | | usage
| | | | yolo_v5
| | | | environments
| | | | tutorials
| | | examples
| | | | YOLOV8-CPP-Inference
| | | | YOLOV8-ONNXRuntime
| | | | YOLOV8-ONNXRuntime-CPP
| | | | YOLOV8-OpenCV-ONNX-Python
| | | runs
| | | | detect
| | | | | train
| | | | | weights
| | | tests
| | | ultralytics
| | | | cfg
| | | | | models
| | | | | | rt-detr
| | | | | | v3
| | | | | | v5
| | | | | | v6
| | | | | | v8
| | | | | trackers
| | | | | __pycache__
| | | | data
| | | | | scripts
| | | | | __pycache__
| | | | engine
| | | | | __pycache__
| | | | hub
| | | | | __pycache__
| | | | models
| | | | | fastsam
| | | | | | __pycache__
| | | | | nas
| | | | | | __pycache__
| | | | | rtdetr
| | | | | | __pycache__
| | | | | sam
| | | | | | modules
| | | | | | | __pycache__
| | | | | | | __pycache__
| | | | | utils
| | | | | yolo
| | | | | | classify
| | | | | | | __pycache__
| | | | | | detect
| | | | | | | __pycache__
| | | | | | pose
| | | | | | | __pycache__

```


main/ 目录详解

这是整个项目的核心模块，包含了 Ultralytics YOLOv8 的源码结构、数据集、文档、部署示例、测试脚本等，功能非常完整。

1. main/ultralytics/ (核心库源码)

这是 YOLOv8 官方代码的本地复制或修改版，支持定制与深度开发：

- `cfg/`：模型配置文件，如 `v3`, `v5`, `v8`, `rt-detr` 等不同模型结构。
- `models/`：模型定义模块，如 `yolo`, `fastsam`, `sam`, `rtdetr`, `nas` 等。
- `nn/`：神经网络结构定义模块，包含模块层（如卷积、注意力等）。
- `engine/`：模型推理与训练引擎。
- `trackers/`：目标跟踪模块。
- `utils/`：工具函数，如回调函数、辅助模块等。
- `yolo/`：YOLOv8 的主要结构与功能代码。

✅ 功能总结：用于模型训练、推理、部署、跟踪、多任务学习的主要框架源码。

2. main/datasets/

- 包含数据集（例如 `jiaotong`），有标准 YOLO 格式的 `images/` 与 `labels/` 子目录，适用于训练和验证。

✅ 功能总结：项目使用的数据集目录，可能为交通相关目标检测数据。

3. main/runs/

- 记录 YOLO 的训练结果，如 `weights/` 里保存模型训练生成的 `.pt` 文件。

✅ 功能总结：YOLO 的训练输出目录。

4. main/examples/

- 包含多种部署和推理示例：
 - `YOLOv8-ONNXRuntime`
 - `YOLOv8-OpenCV-ONNX-Python`
 - `YOLOv8-CPP-Inference`

✅ 功能总结：跨平台模型部署与推理示例，涵盖 C++ 和 Python 环境。

5. main/docs/

- 包含帮助文档、使用教程、模型结构图解、集成说明等。
- 子目录如 guides/, hub/, models/, reference/ 等说明详细全面。

✅ 功能总结: 完整的开发文档与参考资料, 适合开发者查阅。

6. main/tests/

- 测试脚本, 用于验证训练或部署代码正确性。

✅ 功能总结: 项目测试相关内容。

7. main/docker/

- 存在 Dockerfile, 则说明提供了容器化部署的能力。

✅ 功能总结: 支持 Docker 化部署。

youi/ 目录详解

- components/: 自定义组件 (可能为界面或数据组件)。
- ui/: 用户界面相关, 包括图像资源 (assets)、界面组件。
- data/ 和 utils/: 辅助数据与工具代码。

✅ 功能总结: 前端界面模块, 可能用于展示检测结果或交互操作, 如上传图片、可视化 bounding box 等。

✅ 总结: 项目结构总览

目录名	功能说明
.idea/	IDE 配置
main/	🔥 主项目目录, 含 YOLOv8 核心源码、数据集、文档、部署示例等
main/ultralytics/	YOLOv8 源码本地副本, 供深度定制
main/datasets/	训练数据集目录, YOLO 格式
main/examples/	推理部署样例, 支持 ONNX、CPP、OpenCV 等
main/docs/	项目文档和 API 指南
runs/ & main/runs/	模型训练输出, 如权重、结果图等
youi/	用户交互界面模块, 可能为 Web 或桌面可视化
docker/	容器化部署支持 (如有)

注意



此封面在此次，替换掉封面即可。

